



From AI Assistants to Secure by Design: A Multi-Episode SE Radio Digest

Karthik Vaidhyanathan

From the Editor

If you've been following SE Radio lately, you'll have noticed a common thread—software engineering is evolving fast, and we're all trying to keep up. In this column, we provide a round-up of our recent episodes. Over the past few weeks, we've heard from Luke Hinds on keeping AI coding assistants safe, Duncan McGregor and Nat Pryce on the journey from Java to Kotlin, Artie Shevchenko on why programmers should be guardians of code health, and Dan Bergh Johnson and Daniel Deogun on building security into design from day one. Each of these conversations offers something practical that you can take back to your own projects, whether it's a security tip, a migration strategy, or a mindset shift. And together, they paint a picture of where the craft of software engineering is heading: faster, smarter, but also more thoughtful. To hear the full interview, visit <http://www.se-radio.net> or access our archives via RSS at <http://feeds.feedburner.com/se-radio>.—Robert Blumen and Karthik Vaidhyanathan

Episode 680: Luke Hinds on Privacy and Security of AI Coding Assistants

Host: Brijesh Ammanath

Guest: Luke Hinds, CTO of Stacklok and Creator of Sigstore

Luke Hinds, CTO of Stacklok^a and creator of Sigstore, joined host Brijesh

^a<https://stacklok.com/>.

Ammanath to discuss security risks in AI coding assistants. Hinds traced the evolution from the 2017 “Attention Is All You Need” paper through GitHub Copilot’s^b “fill in the middle” autocomplete functionality, which allowed models to predict code based on surrounding context.

Hinds identified data exfiltration as the primary threat. AI assistants need context to function, so they capture entire files from IDEs

^b<https://github.com/features/copilot>.

and transmit them to remote endpoints. Hardcoded application programming interface (API) tokens, database passwords in .env files, and accidentally saved documents all get sent along. While Hinds acknowledged that major providers likely won't exploit leaked secrets, he warned that these data enter training pipelines, and models can later regurgitate what they've learned. Hinds described stale training data as another critical vulnerability. Models trained months ago don't

SOFTWARE ENGINEERING RADIO

Visit www.se-radio.net to listen to these and other insightful hour-long podcasts.

RECENT EPISODES

- 688—Daniel Stenberg, Swedish Internet protocol expert and founder and lead developer of the Curl project, speaks with SE Radio host Gavin Henry (<https://se-radio.net/team/gavin-henry/>) about removing Rust from Curl.
- 686—François Daoust, W3C staff member and cochair of the Web Developer Experience Community Group, discusses with host Jeremy Jung the origins of the W3C, the browser standardization process, and how it relates to other organizations like TC39, WHATWG, and IETF.
- 685—Will Wilson, CEO and cofounder of Antithesis, explores Deterministic Simulation Testing (DST) with host Sriram Panyam (<https://se-radio.net/team/sriram-panyam/>).
- 681—Qian Li of DBOS, a durable execution platform born from research by the creators of Postgres and Spark, speaks with host Kanchan Shringi (<https://se-radio.net/team/kanchan-shringi/>) about building durable, observable, and scalable software systems.

UPCOMING EPISODES

- Amey Desai explores the Model Context Protocol (MCP) and its role in enabling agentic AI systems with host Sriram Panyam.
- Kacper Łukawski, a senior developer advocate from Qdrant, speaks with host Gregory M. Kapfhammer about Qdrant, a vector database and similarity search engine.
- Philip Kiely, a software developer relations lead at Baseten, speaks with host Jeff Doolittle about multi-agent AI.
- Flavia Saldana, a consulting data engineer, joins host Kanchan Shringi to discuss the evolution of data engineering from ETL and data lakes to modern lakehouse architectures enriched with vector databases and embeddings.

know about newly discovered vulnerabilities or compromised packages. He illustrated this with Log4J^c: a model trained before its discovery could recommend the vulnerable library for months afterward. This isn't theoretical, Hinds explained; Stacklok discovered a malicious

^c<https://logging.apache.org/log4j/2.x/index.html>.

npm package used by North Korean threat actors in fake job interviews that would backdoor developers' machines. Months later, he noted, frontier models still recommended it with installation instructions.

On typosquatting attacks, Hinds explained how neural networks calculate similarity through vectorized weights. When given slightly misspelled package names like

“requestz” instead of “requests,” models often accept the typo because they find close proximity patterns, similar to why they struggle answering “how many R's in strawberry.” Hinds warned against “vibe coding,” in particular, auto-approving AI-generated code without scrutiny. He urged developers to watch for insecure patterns like unsanitized input and use proper secret management systems like Vault or hardware security modules rather than hard-coding credentials. He noted that existing protections like .gitignore stop repository commits, but these don't prevent IDE exfiltration.

Hinds introduced Codegate,^d Stacklok's open source solution that runs locally and intercepts traffic between IDEs and large language model (LLM) endpoints. He explained that it uses machine learning to detect secrets and personally identifiable information, redacts them with placeholder strings before transmission, and then swaps back the originals when responses return. Developers see their actual tokens, but nothing sensitive leaves their machine. Code Gate also flags malicious packages using similarity search against Stacklok's threat database. Hinds stressed that security must be seamless; if it hinders productivity, developers won't adopt it, citing his experience with Sigstore, where ease of use drove adoption.

Looking ahead, Hinds expressed concern about Model Context Protocol, which allows LLMs to control systems like Google Drive, GitHub, and Jira. Combined with autonomous agents, he warned, LLM malfunctions, such as catastrophic forgetting and overfitting, could have serious consequences as these

^d<https://github.com/stacklok/codegate>.

systems gain more control over critical infrastructure.

You should listen to this episode if your team is using or considering AI assistants and you want a balanced perspective. Hinds provides a practical road map for enjoying their benefits while securing your systems, reminding us that speed means little if it compromises security.

Episode 682: Duncan McGregor and Nat Pryce on Refactoring from Java to Kotlin

Host: Giovanni Asproni

Guests: Duncan McGregor and Nat Pryce, Coauthors of *Java to Kotlin: A Refactoring Guidebook*

Duncan McGregor and Nat Pryce, coauthors of *Java to Kotlin: A Refactoring Guidebook*, discussed their experience migrating Java codebases to Kotlin with host Giovanni Asproni. Both have used Kotlin extensively in production environments, including at a scientific publisher, where they struggled with functional programming in Java before adopting Kotlin as a more viable alternative.

McGregor emphasized that technically, teams should “just do it”—Kotlin adds no weight to Java programs. Pryce explained that JetBrains designed Kotlin for gradual adoption. He mentioned that one can add it to the build without changing anything, write bits of Kotlin, and have seamless interop. The compiler inserts null checks at the interop layer, gradually increasing type safety. JetBrains provides a keypress that converts Java files to Kotlin instantly, with compatibility options for the remaining Java code. The speakers discussed Kotlin’s “grain” and how languages push developers toward certain patterns. Pryce noted that Kotlin defaults to immutable collections and nonnullable

types, making functional programming natural. At a scientific publisher, they removed hundreds of lines per domain model class using Kotlin’s data classes. Java’s grain has been shaped by Spring⁶ with annotations and reflection. Pryce stressed this isn’t a big bang rewrite. Small, safe changes let teams continue delivering features while converting. They actually saw delivery accelerate as they converted more code to Kotlin. McGregor noted that while AI tools could handle larger transformations, Pryce cautioned that massive changes would conflict with team integration and require extensive review.

On language differences, Pryce highlighted Kotlin’s superior type system. Every type is nonnullable by default, something Java’s Project Valhalla won’t deliver for years. Kotlin eliminates primitive versus reference type distinctions, automatically handling boxing. McGregor pointed to inline functions as particularly powerful, allowing custom control structures without reflection. They discussed AI coding assistants. McGregor noted that JetBrains’ local AI autocomplete proved surprisingly capable. Pryce suggested that traditional refactoring tools remain valuable alongside LLMs that generate code with AI and then refactor it to match your style. McGregor envisioned AI tools eventually using traditional refactoring operations as building blocks.

For business considerations, Kotlin was introduced quietly because seamless Java interop meant no productivity overhead. McGregor noted that Java programmers read Kotlin from day one and write it by day two. The IDE teaches idiomatic patterns through inspection hints. They started with data models where Kotlin’s concise syntax provided immediate value.

⁶<https://spring.io/>.

On performance, Pryce explained that while Kotlin may introduce some boxing, teams can drop down to Java virtual machine-specific optimizations when needed, just as in Java. McGregor added that inline functions and value classes enable performance gains. Both noted that modern Java versions have improved significantly, so teams happy with recent Java might not need Kotlin, though they find Kotlin “more fun.”

You should listen to this episode if you’re considering Kotlin adoption or are interested in gradual language migration. McGregor and Pryce provide practical and experience-based guidance on maintaining team velocity while improving code quality through incremental transformation.

Episode 683: Artie Shevchenko on Programmers as Code Health Guardians

Host: Jeff Doolittle

Guest: Artie Shevchenko, Author of *Code Health Guardian*

Artie Shevchenko, a former Google software engineer and author of *Code Health Guardian*, discussed with host Jeff Doolittle why managing code complexity will be programmers’ crucial role in the AI era. Shevchenko argued that while collaborating with and delegating to AI are important, programmers must excel at something AI struggles with, such as complex reasoning and keeping codebases reasonably simple.

Shevchenko defined code health pragmatically as a productivity question, emphasizing the balance between short-term and long-term productivity. He explained that focusing too much on long-term productivity leads to endless planning without delivery, while

excessive short-term focus creates complexity that eventually halts progress. His startup experience illustrated both extremes: back-end engineers over-invested in architecture, while front-end engineers built frameworks unnecessarily yet later found themselves unable to make simple changes because the code had become incomprehensible.

Shevchenko stressed the importance of having an ownership mentality. When it was his own money funding

configurations because automation should have handled it, not realizing that old jobs lacked that automation. The jobs failed spectacularly, choosing nodes across continents and attempting massive data transfers. He outlined seven complexity causes: obscurity, complex interfaces, unfamiliarity, too many dependencies, duplication, unstable dependencies, and information leakage. Shevchenko noted that while these trace back to Ousterhout's depen-

based on an internal research (Project Aristotle) conducted at Google.

You should listen to this episode if you want practical guidance on managing complexity as AI reshapes programming. Shevchenko's experience across Google, startups, and Canva provides concrete strategies for becoming what he calls a "code health guardian"—and the essential human role in the AI era.

Episode 684: Dan Bergh Johnsson and Daniel Deogun on *Secure by Design*

Host: Sam Taggart

Guests: Dan Bergh Johnsson, Partner at Omega Point, and Daniel Deogun, Chief Academy Officer at Omega Point; Coauthors of *Secure by Design*

Dan Bergh Johnsson and Daniel Deogun, authors of "Secure by Design,"^f discussed with host Sam Taggart how good design practices naturally address security concerns. Bergh Johnsson stated that this captures 95% of security intent, with the remaining 5% requiring awareness of specific security practices and procedures.

Deogun explained that developers focus on delivering features and meeting timelines under pressure from tech and product leads. He argued that developers can start writing secure software by applying good design practices, getting security "not for free, but for a lot cheaper than otherwise." Bergh Johnsson described a problematic gap between traditional development and security roles. He noted meeting senior architects who take responsibility for uptime and maintainability but defer security to specialists, saying, "I'm not a security person." Deogun added that this

^f<https://www.manning.com/books/secure-by-design>.

On typosquatting attacks, Hinds explained how neural networks calculate similarity through vectorized weights.

development, he cared intensely about productivity. He advocated for engineers to think like product owners, noting that distributed responsibility between product managers and engineers creates problematic push-pull dynamics. Engineers have the full picture since they understand both user needs and technical debt, thereby making them best positioned to balance short and long-term concerns.

Shevchenko identified three complexity problems, as also presented in the book. Change amplification occurs when small changes ripple across hundreds of lines and dozens of files. Cognitive load represents the effort required to understand code. But the third one, which is about unknown unknowns, corresponding to missing information one doesn't even know is needed, is the most severe. He recounted a Google Maps incident where a senior engineer deleted batch job

dencies and obscurity, being more specific proves pragmatically useful.

On design principles, Shevchenko emphasized design by contract, which is about trusting interfaces through explicit documentation and internal enforcement via tests and assertions. He challenged strict "Don't repeat yourself" (DRY) adherence, arguing that unknown unknowns around duplicates matter more than change amplification. If duplicates are documented and known, changing them in multiple places is manageable. For processes, Shevchenko advocated design documents for upfront tradeoff analysis, cautioning that once code is implemented and merged, no one wants to discuss alternatives. He identified four code review goals: quality, knowledge sharing, delivery speed, and psychological safety—the last being critical for team productivity

creates an organizational mindset where security is “somebody else’s problem.” Bergh Johnsson drew parallels to early agile, when developers and testers were separate, with sometimes antagonistic relationships. He explained that throughout several years, coding and testing merged into the same cycle, and now security is being embraced within that cycle rather than treated separately. The speakers emphasized domain-driven design’s security benefits. Deogun explained that developers commonly fall back on language primitives, such as strings, integers, floats, etc., to represent specific business concepts. Using an integer for quantity accepts all integer capabilities: negative numbers and extremely large numbers. From a business perspective, neither negative quantities nor a billion books makes sense, but from a software perspective, it works. Bergh Johnsson recounted public bugs and client assessments where negative amounts multiplied by positive prices created discounts. Further, he described a client they had at one point; there was an addition where one bus number was added to another bus number (which was not the intent), but someone had just mistaken two parameters for each other and ended up adding two bus numbers, coming up with something that looked like a bus number, but in the end, made no sense at all and led to very strange behavior.

They detailed a five-step validation order. First, check the origin of data—do they come from a valid place? Second, check size or length, and this is a quick check to perform. Third, check if it contains proper characters without caring about order. Fourth, parse the data to check character order. This might be more expensive because you open the package and look inside. Fifth, perform semantic

validation checks if it makes sense at this point in time, and this is the most expensive, typically requiring database queries. Bergh Johnsson credited security researcher Dr. John Wilander for phrasing this specific ladder.

On logging, Deogun explained second-order attacks where attackers inject vectors not targeting the primary system but logged for interpretation by tools with vulnerabilities. He argued against logging full inputs, suggesting instead logging that you got invalid input describing the problem. Deogun also highlighted that developers forget to limit privileges for log-writing processes, recommending that systems write-only to logs while humans read-only. They also then went on to mention the three R’s in the book, which are *rotate*, *repave*, and *repair*. Bergh Johnsson explained repave as replacing running instances on a structured schedule, constantly replacing everything with fresh instances. This forces attackers to reinstall, creating detectable network traffic. Deogun discussed rotation—designing software to fetch secrets from key vaults rather than hardcoding them. Bergh Johnsson mentioned a client who hadn’t changed their production database password in 18 years because their application design made it too complicated. Finally, Deogun explained repair as rebuilding software

and images, upgrading to the latest dependency versions, then deploying in the normal repavement cycle.

Bergh Johnsson concluded that culture is fundamental for technical work. He explained that organizations promoting high-trust environments where concerns can be voiced and discussed openly display higher security. He stated, “Technical excellence without a culture is not possible. And cultural excellence without any technical knowledge at all will not yield security.”

You should listen to this episode if you want to understand how deliberate design patterns create security outcomes accessible to all developers.

Each of these episodes highlights a different challenge in modern software engineering: from using AI assistants safely to evolving codebases with Kotlin, to protecting code health, to building security in from the start. What ties them together is the reminder that good engineering requires foresight. Small decisions today shape the resilience, safety, and speed of tomorrow’s systems. To hear the full depth of these conversations the anecdotes, the detailed practices, and the insights that don’t fit on the page, head over to se-radio.net (<https://se-radio.net>). 🎧

ABOUT THE AUTHOR



KARTHIK VAIDHYATHAN is an assistant professor at the Software Engineering Research Center, IIIT-Hyderabad, Hyderabad 500032, India. Contact the author at karthik.vaidhyathan@iiit.ac.in